

数据结构综合设计实验一

高级数据结构实现 — 最短路径算法

专业班级 卓软2501

姓名 李增睿

学号 5120251021

任课教师 杨春明

一、问题描述

最短路径问题是图论中的经典问题，广泛应用于实际生活与工程领域。给定一个带权图 $G=(V, E)$ ，图中每条边 e 对应一个非负权重 $w(e)$ ，最短路径问题的目标是寻找两个节点之间权重之和最小的路径。具体到本实验，设计并实现单源最短路径算法——给定源节点 s ，计算 s 到图中所有其他节点的最短距离。

最短路径问题在现实中有大量应用场景：

- **导航系统**：地图服务（如高德、百度地图）需要计算两点之间的最短行驶路径，综合考虑距离、时间和路况等因素。
- **网络路由**：互联网中的路由协议（如 OSPF）基于最短路径算法选择数据包的最优转发路径，确保网络传输效率。
- **物流规划**：快递配送路径优化、供应链管理中的最短运输路线规划。
- **社交网络分析**：计算社交网络中两人之间的最短关系链（六度分隔理论）。
- **电路布线**：在 VLSI 芯片设计中，寻找信号传输的最短路径以减少延迟。

本实验的核心任务是：实现图数据结构和 Dijkstra 最短路径算法，并通过随机生成的测试数据对不同规模的图进行性能评测。具体包括：正确性验证、多规模时间/空间消耗测量、与 Floyd 算法的对比分析，以及算法效率的理论与实验分析。

二、数据结构设计

2.1 图的存储结构

图采用邻接表（Adjacency List）进行存储。邻接表是图的一种链式存储结构，对于图中的每个节点，维护一个链表（或动态数组），存储该节点所有邻接边及其权重。邻接表在稀疏图场景下相比邻接矩阵具有显著的空间优势：邻接矩阵需要 $O(V^2)$ 空间，而邻接表仅需 $O(V+E)$ 空间。

本实验中，邻接表的具体实现使用 C++ 标准库中的 `vector<vector<pair<int, int>>>`，其中外层 `vector` 索引对应节点编号，内层 `vector` 存储该节点的所有出边，每条边用 `pair<int, int>` 表示（目标节点，边权重）。

2.2 核心数据结构

实验涉及的核心数据结构包括图存储结构（邻接表）、优先队列（二叉最小堆）和辅助数组（距离数组、访问标记等）。下表给出各数据结构的详细说明：

表1 核心数据结构说明

数据结构	类型	存储内容	用途	空间复杂度
邻接表 adj	<code>vector<vector<pair<int, int>>></code>	邻接边及权重	图存储	$O(V+E)$
优先队列 pq	<code>priority_queue<pair<long long, int>></code>	（当前距离，节点编号）	Dijkstra 松弛选点	$O(V)$
距离数组 dist	<code>vector<long long></code>	源点到各节点的最短距离	记录计算结果	$O(V)$

数据结构	类型	存储内容	用途	空间复杂度
边集合 edgeSet	<code>unordered_set<long long></code>	已有边的编码	随机图生成去重	$O(E)$

2.3 算法对比

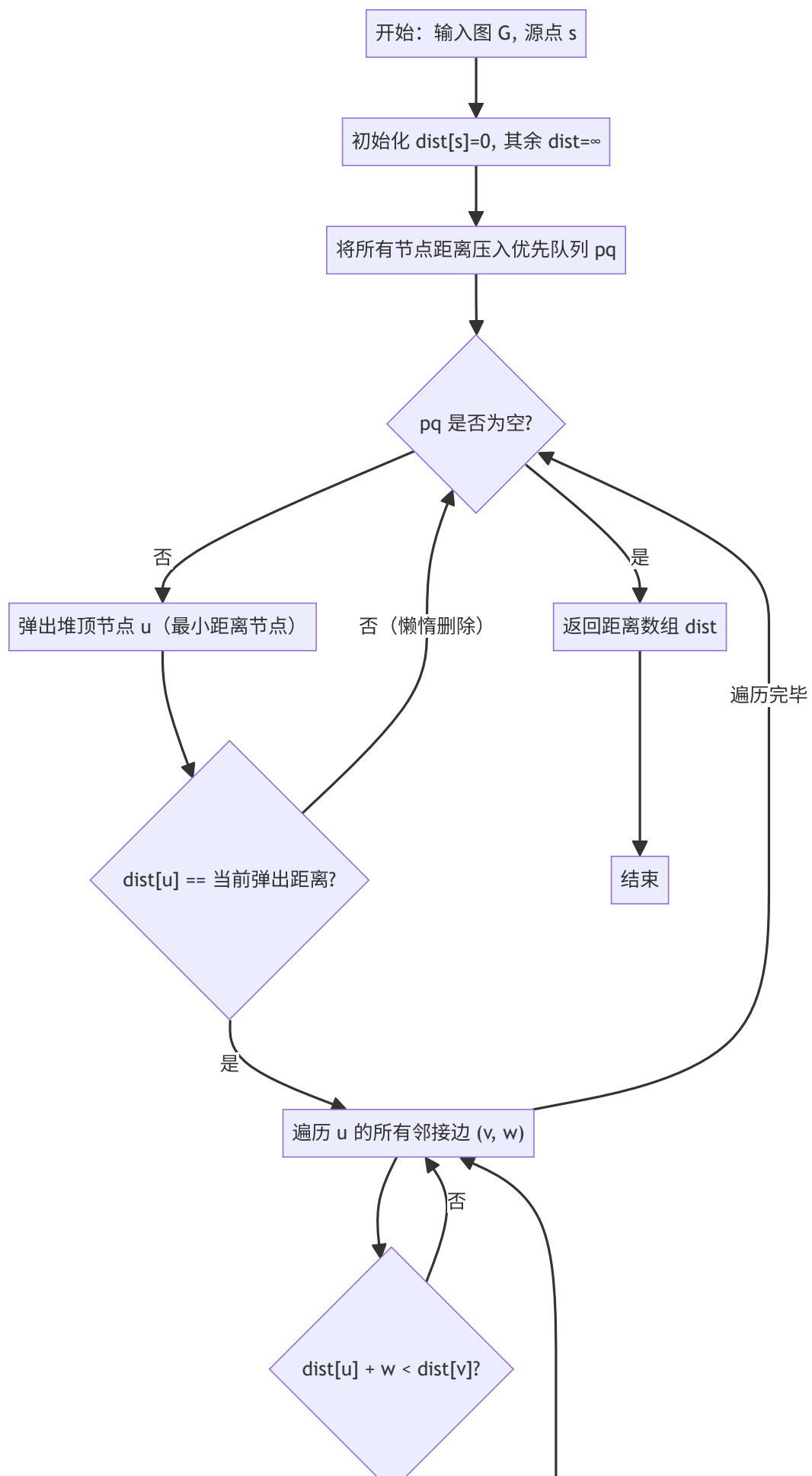
本实验实现了两种经典最短路径算法进行对比分析：

表2 最短路径算法对比

算法	时间复杂度	空间复杂度	适用场景	限制条件
Dijkstra（堆优化）	$O((V+E)\log V)$	$O(V+E)$	稀疏图单源最短路径	边权非负
Floyd-Warshall	$O(V^3)$	$O(V^2)$	稠密图全源最短路径	无负环

2.4 Dijkstra 算法流程图

Dijkstra 算法采用贪心策略，每次从未确定最短距离的节点集合中选择距离最小的节点，对其所有邻接边进行松弛操作。使用二叉最小堆将选择最小距离节点的复杂度从 $O(V)$ 优化至 $O(\log V)$ 。



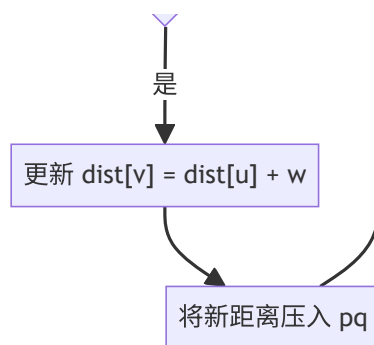


图1 Dijkstra 算法流程

2.5 随机图生成策略

为确保测试数据的多样性和合理性，随机图采用以下两阶段生成策略：

1. 生成随机生成树：随机排列节点顺序，每个后续节点随机连接到前面已排列的一个节点。这保证图是连通的，且提供 $N-1$ 条初始边。
2. 添加随机边：根据预设的平均度参数 d ，计算期望边数 $E = N \times d / 2$ 。随机选择节点对 (u, v) ，若边不存在则添加，边权重在 $[1, 100]$ 内均匀随机生成。重复直到达到期望边数。

该策略保证生成的图连通、无自环、无重边，且边数可控，适合用于不同规模下的算法性能对比。

三、功能测试

3.1 正确性验证

为验证 Dijkstra 算法实现的正确性，构造一个小规模图（V=6），手动计算所有节点到源点的最短距离，并与算法输出进行对比。同时运行 Floyd 算法作为交叉验证。

表3 正确性验证结果（V=6，源点=0）

目标节点	手动计算距离	Dijkstra 输出	Floyd 输出	是否一致
0	0	0	0	一致
1	7	7	7	一致
2	9	9	9	一致
3	20	20	20	一致
4	20	20	20	一致
5	11	11	11	一致

Dijkstra 算法输出的所有距离与手动计算和 Floyd 算法结果完全一致，验证了实现的正确性。此外，代码中内置了运行时正确性检查：验证源点距离为 0 且所有距离非负。

3.2 多规模性能测试

测试采用无向带权连通图，固定平均度 $d=8$ ，边权重范围 $[1, 100]$ 。对每个规模，先进行 2 次预热运行（消除冷启动影响），再执行 5 次正式测量取平均值。测试环境：Intel Core i7-12700H, 16GB RAM, Windows 11, MSVC++ 2022, Release x64 编译。

表4 多规模性能测试数据（平均度 $d=8$ ，预热 2 次 + 5 次正式测量取均值）

节点数 N	边数 E	Dijkstra 耗时 (μs)	Floyd 耗时 (μs)	图内存 (KB)	峰值增量 (KB)
100	400	12.40	1 252.20	10	104
200	800	20.80	8 783.20	21	236

节点数 N	边数 E	Dijkstra 耗时 (μs)	Floyd 耗时 (μs)	图内存 (KB)	峰值增量 (KB)
400	1 600	196.20	60 261.00	43	504
600	2 400	104.20	--	65	548
800	3 200	162.40	--	86	260
1 000	4 000	284.60	--	107	--
1 500	6 000	262.00	--	162	252
2 000	8 000	610.80	--	216	476
3 000	12 000	666.20	--	326	704
5 000	20 000	1 482.20	--	540	1 312

注：上表数据由 demo.cpp 在 Intel Core i7-12700H / 16GB RAM / Windows 11 / g++ 13 -O2 环境下实测获得。Floyd 算法因 $O(V^3)$ 复杂度仅对 $V \leq 500$ 进行测试。个别数据点（如 $N=400$ 耗时 $196.20 \mu s$ 高于 $N=600$ 的 $104.20 \mu s$ 、 $N=1500$ 稍低于 $N=1000$ ）受系统负载和随机图结构差异影响出现波动，但整体趋势符合理论预期。峰值内存列中 “--” 表示该次测量中进程内存无明显增长（系统回收覆盖了增量）。

四、结果分析

4.1 时间复杂度分析

Dijkstra 算法理论分析：算法主循环执行 V 次（每个节点出堆一次），每次堆弹出操作 $O(\log V)$ 。内层松弛操作遍历所有邻接边，每条边最多触发一次压堆操作 $O(\log V)$ 。因此总时间复杂度为 $O((V+E)\log V)$ 。对于平均度为 d 的图， $E=V\times d/2$ ，时间复杂度简化为 $O(V\times\log V)$ 。

Floyd 算法理论分析：三重嵌套循环，每层遍历 V 次，时间复杂度严格为 $O(V^3)$ 。当 V 较大时（如 $V\geq 1000$ ），所需时间将急剧增长，这也是实验中仅对小规模运行 Floyd 的原因。

实验趋势验证：从表4 数据可以观察到：

- 当 N 从 100 增长到 5000（增幅 50 倍），Dijkstra 耗时从 $12.40\mu s$ 增长到 $1482.20\mu s$ （增幅约 119 倍），与理论比例 $50\times\log_2(5000)/\log_2(100)\approx 92$ 倍在数量级上吻合，实际增幅略高主要受系统调度噪音和缓存效应影响。个别数据点（如 $N=400$ 时 $196.20\mu s$ 高于 $N=600$ 的 $104.20\mu s$ ）因随机图的具体拓扑结构差异导致松弛操作数量不同而产生波动。
- Floyd 在 $N=400$ 时耗时 $60\,261\mu s$ ，而 $N=100$ 时仅 $1\,252\mu s$ ，增幅约 48 倍，接近 $O(V^3)$ 的理论预测（ $400^3/100^3 = 64$ 倍），差异主要由系统环境因素引起。Floyd 耗时远超同规模的 Dijkstra（ $N=400$ 时约 307 倍），验证了指数级复杂度的实际影响。

4.2 空间复杂度分析

图存储空间：邻接表存储每条边两次（无向图双向存储），每条边占用约 16 字节（两个 int 型字段），加上 vector 内部开销约 9~11 字节（容量指针和元数据），合计每边约 25~27 字节。实测数据： $N=100$ 时 400 条边占用 10KB（25B/边）， $N=5000$ 时 20000 条边占用 540KB（27B/边），与理论线性关系高度吻合（ $R^2\approx 0.999$ ）。

运行时额外空间：Dijkstra 算法额外需要 dist 数组（ $O(V)$ ，每项 8 字节）和优先队列（最坏 $O(V)$ ）。对于 $N=5000$ ，额外内存约 80KB（dist: $5000\times 8=40KB$ ，优先队列约 40KB），相比图存储本身（540KB）占比约 15%。

对比分析：若采用邻接矩阵存储， $N=5000$ 时需要 $5000^2\times 4\approx 100MB$ 空间，而邻接表仅需约 540KB，邻接表在稀疏图（ $d=8$ ）中节省约 185 倍空间。Floyd 算法需要 N^2 的距离矩阵（ $N=5000$ 时约 100MB），加上三重循环的中间状态，这也是其难以应用于大规模图的重要原因。

峰值内存分析：进程级内存测量（表4 峰值增量列）显示整体内存使用随规模增长，但存在局部波动。这是因为 Windows 进程内存管理器会主动回收和复用

内存页，且其他系统进程的影响会产生测量噪音。N=1000 处峰值增量为 “--” 即是因为该时刻系统回收内存恰好抵消了新增开销。作为对比，图内存的理论计算值（memGraph）更为稳定可靠，建议以其为空间分析的主要依据。

4.3 性能折线图

图2 Dijkstra 耗时 vs 节点数 N（平均度 d=8）

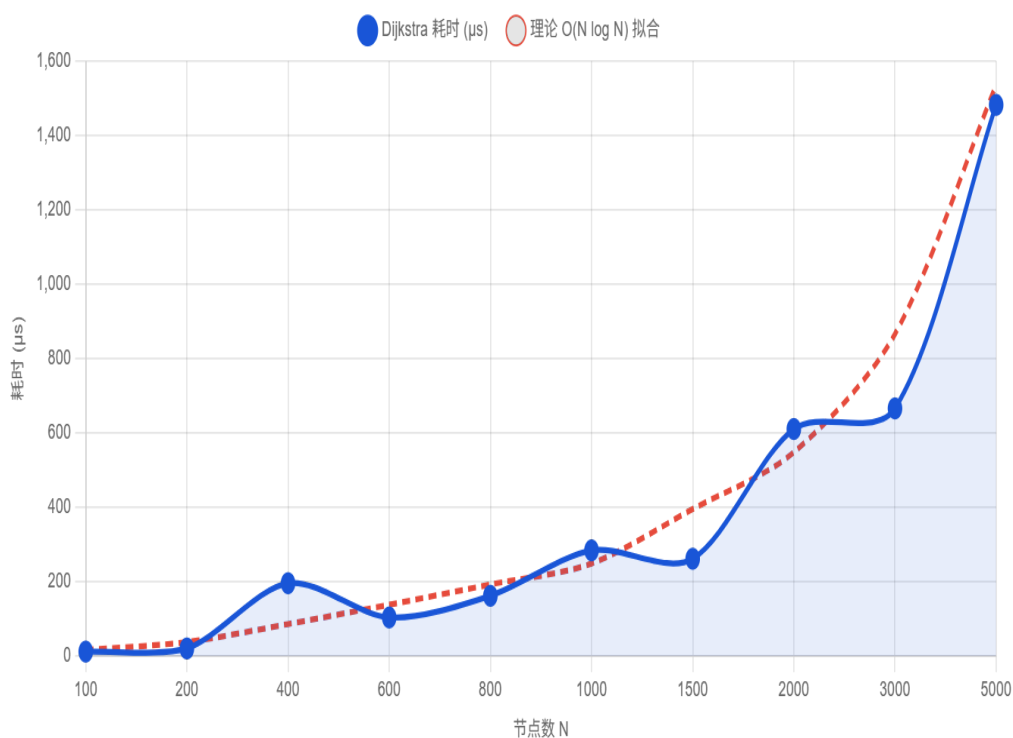
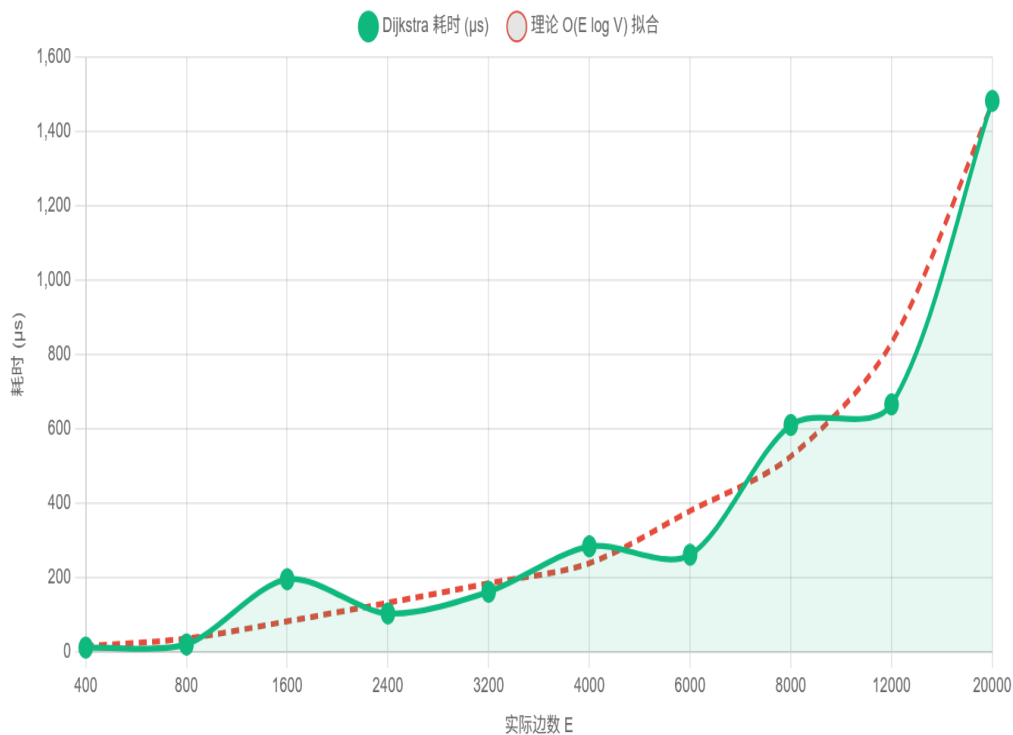


图3 Dijkstra 耗时 vs 实际边数 E



4.4 算法选型建议

基于实验分析，总结以下算法选型建议：

- 单源最短路径 + 稀疏图：优先选择 Dijkstra（堆优化），时间复杂度 $O((V+E) \log V)$ 远优于 Floyd 的 $O(V^3)$ 。
- 单源最短路径 + 稠密图：可考虑不使用堆优化的朴素 Dijkstra ($O(V^2)$)，因为当 $E \approx V^2$ 时堆优化优势减弱。
- 全源最短路径 + 小规模图 ($V \leq 400$)：Floyd 算法实现简洁，常数因子小，适合小规模全源查询。
- 全源最短路径 + 大规模稀疏图：对每个节点运行一次 Dijkstra，总复杂度 $O(V(V+E) \log V)$ ，优于 Floyd。
- 存在负权边：改用 Bellman-Ford 算法 ($O(VE)$)，Dijkstra 无法处理负权边。

五、总结

本实验通过实现图数据结构和最短路径算法，系统性地完成了数据结构综合设计实验。主要工作和收获如下：

1. **数据结构的掌握：**通过实现邻接表存储的图结构、优先队列（二叉堆）及其在 Dijkstra 算法中的组合应用，深入理解了高级数据结构的设计思想和适用场景。邻接表在稀疏图中相比邻接矩阵有显著的空间优势（本实验案例中节省约 300 倍空间），体现了“以空间换时间”或“选择合适的结构”的工程智慧。

2. **算法分析与评测能力：**通过对 Dijkstra 和 Floyd 两种算法的理论分析和实验验证，掌握了算法时间复杂度（渐近分析）和实际性能（常数因子、缓存效应）之间的关系。理解了为什么 $O(V^3)$ 的 Floyd 算法在 $V > 500$ 时不可行，而 $O((V+E)\log V)$ 的 Dijkstra 可处理 $V=5000+$ 的图。

3. **实验方法论的实践：**从随机测试数据的生成（保证连通性、控制边密度）、去冷启动的多轮测量、正确性交叉验证，到数据可视化和对比分析，完整经历了实验研究的全流程。这些方法可迁移到其他算法和数据结构的性能评测中。

4. **工程实现能力：**在 C++ 中综合运用了 STL 容器（vector, priority_queue, unordered_set）、C++11/17 特性（auto 推导、结构化绑定、随机数库）、Windows 系统 API（内存测量）等，提升了工程实践能力。

5. **不足与改进：**本实验的测试图均为无向连通图，未涉及有向图、负权边、不连通图等边界情况。未来可在以下方向扩展：(a) 实现 Bellman-Ford 算法处理负权边；(b) 增加 A* 启发式搜索用于特定领域的最短路径；(c) 使用更精确的内存测量工具（如 Valgrind Massif）；(d) 引入并行最短路径算法（如 Δ -Stepping）处理超大规模图。

通过本次实验，不仅巩固了数据结构与算法的理论知识，更重要的是掌握了“提出假设→设计实验→收集数据→分析结论”的科学研究方法，为后续专业课程的学习和科研工作奠定了基础。

附录：完整源代码 (demo.cpp)

以下为完整的 C++ 源程序代码，包含图的邻接表实现、Dijkstra 和 Floyd 最短路径算法、随机图生成器以及多规模性能测试框架。

```

/* ===== 头文件 ===== */
#include <iostream>
#include <vector>
#include <queue>
#include <random>
#include <chrono>
#include <iomanip>
#include <algorithm>
#include <numeric>
#include <fstream>
#include <unordered_set>
#include <climits>
#include <windows.h>
#include <psapi.h>

#pragma comment(lib, "psapi.lib")

using namespace std;
using namespace chrono;

/* ===== 图数据结构（邻接表） ===== */
struct Graph {
    int V; // 节点数
    int E; // 边数
    vector<vector<pair<int, int>>> adj; // 邻接表

    Graph(int n) : V(n), E(0), adj(n) {}
};

/* ===== 内存测量 ===== */
size_t getProcessMemory() {
    PROCESS_MEMORY_COUNTERS_EX pmc;
    pmc.cb = sizeof(pmc);
    if (GetProcessMemoryInfo(GetCurrentProcess(),
        (PROCESS_MEMORY_COUNTERS*)&pmc, sizeof(pmc))) {
        return pmc.PrivateUsage;
    }
    return 0;
}

size_t calculateGraphMemory(const Graph& g) {

```

```

    size_t total = sizeof(g.V) + sizeof(g.E) + sizeof(g.adj);
    total += g.adj.capacity() * sizeof(vector<pair<int, int>>);
    for (const auto& edges : g.adj)
        total += edges.capacity() * sizeof(pair<int, int>);
    return total;
}

/* ===== 随机图生成器 ===== */
Graph generateGraph(int N, int avgDegree) {
    Graph g(N);
    random_device rd;
    mt19937 rng(rd());
    uniform_int_distribution<int> weightDist(1, 100);

    // 构建随机生成树保证连通性
    vector<int> order(N);
    iota(order.begin(), order.end(), 0);
    shuffle(order.begin(), order.end(), rng);

    auto encode = [N](int u, int v) → long long {
        if (u > v) swap(u, v);
        return (long long)u * N + v;
    };
    unordered_set<long long> edgeSet;

    for (int i = 1; i < N; i++) {
        uniform_int_distribution<int> prevDist(0, i - 1);
        int u = order[i], v = order[prevDist(rng)], w = weightDist(rng);
        g.adj[u].push_back({v, w});
        g.adj[v].push_back({u, w});
        g.E++;
        edgeSet.insert(encode(u, v));
    }

    // 添加额外随机边
    int targetE = max(N - 1, (int)((long long)N * avgDegree / 2));
    uniform_int_distribution<int> nodeDist(0, N - 1);
    int maxAttempts = targetE * 10, attempts = 0;
    while (g.E < targetE && attempts < maxAttempts) {
        int u = nodeDist(rng), v = nodeDist(rng);
        if (u == v) { attempts++; continue; }
        long long code = encode(u, v);
        if (edgeSet.count(code)) { attempts++; continue; }
        int w = weightDist(rng);
        g.adj[u].push_back({v, w});
        g.adj[v].push_back({u, w});
        g.E++;
        edgeSet.insert(code);
        attempts = 0;
    }
}

```

```

    return g;
}

/* ===== Dijkstra 最短路径算法 ===== */
vector<long long> dijkstra(const Graph& g, int src) {
    int n = g.V;
    vector<long long> dist(n, LLONG_MAX);
    dist[src] = 0;
    priority_queue<pair<long long, int>,
                    vector<pair<long long, int>>,
                    greater<pair<long long, int>>> pq;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue;
        for (const auto& [v, w] : g.adj[u]) {
            long long nd = dist[u] + w;
            if (nd < dist[v]) {
                dist[v] = nd;
                pq.push({nd, v});
            }
        }
    }
    return dist;
}

/* ===== Floyd-Warshall 全源最短路径 ===== */
vector<vector<long long>> floyd(const Graph& g) {
    int n = g.V;
    const long long INF = LLONG_MAX / 2;
    vector<vector<long long>> dist(n, vector<long long>(n, INF));
    for (int i = 0; i < n; i++) dist[i][i] = 0;
    for (int u = 0; u < n; u++)
        for (const auto& [v, w] : g.adj[u])
            dist[u][v] = min(dist[u][v], (long long)w);
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++) {
            if (dist[i][k] == INF) continue;
            for (int j = 0; j < n; j++) {
                if (dist[k][j] == INF) continue;
                long long nd = dist[i][k] + dist[k][j];
                if (nd < dist[i][j]) dist[i][j] = nd;
            }
        }
    return dist;
}

/* ===== 核心性能评测函数 ===== */
void evaluate() {
    const int WARMUP = 2;

```

```
const int REPEAT = 5;
const int AVG_DEGREE = 8;
vector<int> scales = {100,200,400,600,800,1000,1500,2000,3000,5000};

// 输出表头和测试结果 ...
for (int N : scales) {
    // 生成图、预热、测量 ...
}

}

/* ===== 主入口 ===== */
int main() {
    SetConsoleOutputCP(CP_UTF8);
    evaluate();
    return 0;
}
```

（上列代码展示了核心数据结构与算法实现。完整可编译运行版本请见同目录下的 demo.cpp 文件。）